Phase 08 — AWS Cloud (Complete Notes)

> *The same concepts as your VPS — networks, servers, firewalls, DNS — renamed, API-driven, and infinitely rentable.*

---

1. Cloud concepts

Cloud = renting computers and services by the hour/second via an API. AWS ≈ 33% of the market; the concepts transfer to Azure/GCP.

Why companies pay AWS more than a VPS would cost:

- **Elasticity** — 3 servers normally, 30 during a sale, 3 after; pay for what ran.
- **Managed services** — AWS runs the database (backups, patching, failover) so your team doesn't.
- **Reliability primitives** — multiple datacenters, one API call apart.

Mindset shift from VPS: servers become **cattle, not pets** — disposable, recreated from images/scripts, never hand-nursed back to health.

Regions & Availability Zones

```
REGION ap-southeast-1 (Singapore)            ← choose close to your users
 ├── AZ ap-southeast-1a   ← separate datacenter: own power, cooling, network
 ├── AZ ap-southeast-1b   ← a few km away, microsecond links
 └── AZ ap-southeast-1c
```

One AZ can burn down without taking the region: production = **at least 2 AZs**. (VPS comparison: your Hetzner box is one machine in one datacenter — one fire away from gone.)

2. IAM — Identity and Access Management

Who can do what, on which resource. The skill that separates professionals from breach headlines.

- **Users** (humans), **Roles** (assumed by services — an EC2 instance gets a role instead of stored passwords ★), **Policies** (JSON permission documents), **Groups**.
- Root account: enable MFA, then **never use it again**. Daily work = IAM user/SSO with least privilege.

```
{ "Effect": "Allow", "Action": ["s3:GetObject"], "Resource": "arn:aws:s3:::my-bucket/*" }
```

★ Production pattern: your Spring Boot app on EC2 needs S3 access → attach a **role** to the instance; the SDK picks up temporary credentials automatically. **No access keys in config files, ever.**

3. EC2 — Elastic Compute Cloud (your VPS, cloudified)

- **Instance** = VM. Types: `t3.micro` (1 GB, free tier) → huge. **AMI** = the OS image (Ubuntu 24.04).
- **Key pair** = your SSH public key, injected at launch (Phase 4 knowledge).

- **Elastic IP** = a public IPv4 you *reserve*, survives instance stop/start (default public IPs change on stop!). Point DNS at Elastic IPs, not ephemeral ones.
- **User data** = boot script (install docker, start app) — the beginning of cattle thinking.

```
# everything you did on the VPS works identically:
ssh -i key.pem ubuntu@<elastic-ip>
# then: the Phase-7 ritual, docker compose, etc.
```

Security Groups — the cloud firewall

Like ufw, but *outside* the instance, stateful, default **deny all inbound**:

```
SG "web-sg":      allow 80,443 from 0.0.0.0/0 (world),  22 from YOUR-IP/32 only
SG "app-sg":      allow 8080 from web-sg  ← source = another SG, not an IP! ★
SG "db-sg":       allow 5432 from app-sg
```

That SG-references-SG pattern builds the layered architecture (LB → app → DB) without hardcoding IPs. The DB literally cannot be reached except from app instances.

4. VPC — Virtual Private Cloud (your own network)

Everything from Phase 1 becomes something you *design*:

```
VPC 10.0.0.0/16                                (your private address space)
│
├── PUBLIC subnet  10.0.1.0/24 (AZ-a)   ├── PUBLIC subnet  10.0.2.0/24 (AZ-b)
│     NGINX / Load balancer                   (route table → Internet Gateway)
│
├── PRIVATE subnet 10.0.11.0/24 (AZ-a)  ├── PRIVATE subnet 10.0.12.0/24 (AZ-b)
│     app servers, databases                  (no route from the Internet)
│
├── Internet Gateway (IGW)   — the VPC's door to the Internet
├── NAT Gateway (in public)  — lets PRIVATE machines call OUT (apt update, APIs)
│                              without being reachable IN  ← Phase-1 NAT, as a service!
└── Route tables             — "0.0.0.0/0 → IGW" (public) / "0.0.0.0/0 → NAT" (private)
```

- **Public subnet** = its route table points to the IGW and instances get public IPs.
- **Private subnet** = no such route → unreachable from the Internet, period. **Databases and app servers live here.** This is the architectural version of "don't open 5432 in ufw".
- CIDR notation: `/16` = 65k addresses, `/24` = 256. (10.x.x.x — the private range from Phase 1.)

5. RDS — managed PostgreSQL

Instead of running postgres in Docker/systemd, AWS runs it: automated backups + point-in-time recovery, patching, metrics, **Multi-AZ** (synchronous standby in another AZ; automatic failover — DNS name stays the same), **read replicas** (Phase 9).

Setup essentials: private subnets only, `db-sg` allowing 5432 from app-sg, no public access. Spring config changes only in the JDBC URL:

```
jdbc:postgresql://shop-db.xxxx.ap-southeast-1.rds.amazonaws.com:5432/shop
```

Trade-off vs self-hosted: ~2× the raw EC2 price for the same size; you're buying ops labor, backups, and failover. For most teams: worth it.

6. S3 — Simple Storage Service

Object storage: files (objects) in buckets, addressed by key, via HTTP API. **Not a filesystem, not a disk.** 11 nines durability, effectively infinite, pay per GB.

Uses: user uploads, static assets/frontends, backups, log archives. ★ Solves the Phase-9 problem "user uploaded an image to server 1, request hit server 2": files go to S3, all servers see them.

```
aws s3 mb s3://shop-uploads-xyz
aws s3 cp backup.sql.gz s3://shop-backups/     # your Phase-7 backup script, upgraded
aws s3 sync ./dist s3://shop-frontend          # deploy a static frontend
```

Buckets are **private by default — keep them private**; serve public content via CloudFront. ("Leaky S3 bucket" = the most famous cloud breach genre.) App access via IAM role + presigned URLs for user downloads/uploads.

7. CloudFront — CDN

Problem (Phase 1 physics): Dhaka → US server = 250 ms of unavoidable light-speed tax per round trip. CDN answer: cache copies at 400+ **edge locations** worldwide; users hit the nearest edge.

```
user (Dhaka) → edge (Mumbai, 30ms)  → cache HIT → instant
                                    → cache MISS → origin (S3 or your ALB) → cached for next time
```

- Static assets: served ~10× faster, origin barely touched.
- Also fronts APIs: TLS at the edge, DDoS absorption (AWS Shield), even dynamic requests benefit from optimized backbone routing.
- Invalidation: `aws cloudfront create-invalidation --paths "/*"` after deploying new frontend files (or better: hashed filenames + long cache).

8. Route 53 — DNS (Phase 2, productionized)

Hosted zone = your domain's authoritative nameservers, AWS-run. Beyond ordinary records:

- **Alias records** — point the zone apex (example.com) at ALB/CloudFront/S3 by name (solves the "no CNAME at root" problem from Phase 2).
- **Routing policies**: weighted (canary: 5% → new version), latency-based (EU users → eu-west, Asia → ap-southeast), **failover** (health check fails → route to backup site).
- Health checks that actively probe your endpoints.

9. CloudWatch — metrics, logs, alarms

- **Metrics**: CPU, network, RDS connections, ALB request count/latency/5xx (free, automatic).
- **Logs**: ship app/nginx logs to CloudWatch Logs (agent or awslogs docker driver); search across instances.
- **Alarms** → SNS → email/Slack: `ALB 5xx > 10 in 5 min`, `RDS storage < 10%`, `EC2 status check failed`. Alarms also *trigger auto scaling* (Phase 9).
- Your external UptimeRobot (Phase 7) still applies — never watch only from inside AWS.

10. Cost control (the un-skippable lesson)

Free tier ≈ 750 h/mo t3.micro + some RDS/S3 for 12 months. Bills bite via: NAT Gateway (~$32/mo!), forgotten Elastic IPs, Multi-AZ RDS left running, data transfer OUT. **Do immediately:** billing alert at $5 and $20 (CloudWatch billing alarm), tag everything, and destroy lab resources after practice. For labs: one public subnet + t3.micro + free-tier RDS Single-AZ, skip NAT Gateway.

11. Local → VPS → AWS (the course's running comparison)

Concern	Local	VPS (Phase 7)	AWS
Server	your laptop	rented VM	EC2 (or ECS/Lambda)
Public IP	none (NAT)	included	Elastic IP / ALB
Firewall	–	ufw	Security Groups (+NACL)
Network	home router	provider's	**VPC you design**
DNS	/etc/hosts	registrar panel	Route 53
Database	docker postgres	docker/systemd postgres	**RDS managed**
Files	disk	disk	**S3**
TLS	self-signed	certbot	ACM (free, auto-renew) on ALB/CloudFront
Monitoring	–	htop + UptimeRobot	CloudWatch + alarms
Scaling	–	resize (downtime)	ASG/ALB (Phase 9–10)

12. Common mistakes

- Working as root account daily; access keys in git (bots scan GitHub and mine crypto on your card within minutes — real, common).
- Public S3 buckets; databases in public subnets; SG 22 open to 0.0.0.0/0.
- No billing alarms ("the $3,000 surprise").
- Treating EC2 like a pet VPS: hand-configured, unreproducible (→ use user-data scripts; later, images/IaC).
- One AZ for everything, then blaming AWS during an AZ event.
- Choosing a region far from users (latency is physics).

13. Interview questions

1. Region vs AZ? Why deploy across 2+ AZs?
2. IAM user vs role? How should an EC2 app get S3 permissions and why not access keys?
3. Design a VPC for a 3-tier app: subnets, route tables, IGW, NAT — what goes where and why?
4. Security Group vs ufw/NACL? What does "source = another SG" enable?
5. Why RDS over postgres on EC2? What does Multi-AZ actually do during failover?
6. What is S3 (vs a filesystem)? How do multiple app servers share user uploads?
7. How does a CDN reduce latency? What's an edge location, a cache hit ratio?
8. NAT Gateway vs Internet Gateway?

14. LAB (free tier, ~$0 if cleaned up; set billing alarms FIRST)

1. Create account → MFA on root → IAM admin user → **billing alarm**.

2. Launch t3.micro Ubuntu EC2 (default VPC, SG: 22 from your IP, 80/443 world) + Elastic IP → run your Phase-7 ritual + docker compose stack → Route 53 (or your registrar) A record → certbot → your app on AWS.
3. Build a custom VPC per §4 diagram (skip NAT Gateway; 2 public + 2 private subnets). Launch RDS free-tier PostgreSQL in private subnets with db-sg←app-sg. Point your app's JDBC URL at it. Verify you CANNOT reach RDS from your laptop but the app can.
4. S3: bucket for backups; change your Phase-7 cron to `aws s3 cp` (instance role, no keys!).
5. CloudWatch: alarm on EC2 CPU > 80% (test with `stress` package) → email via SNS.
6. **Destroy everything** except what you keep intentionally; check the bill next day.

15. ASSIGNMENT 08 (submit to Claude)

1. Screenshot/describe your VPC layout: CIDR, subnets, route tables. Explain why the RDS instance is unreachable from your laptop — trace the exact mechanism (route table? SG? both?).
2. Paste your app→S3 backup solution and explain how credentials flow with an instance role (what does the SDK actually receive?).
3. Cost exercise: price your Phase-7 VPS stack on AWS (t3.small + RDS db.t4g.micro Multi-AZ + S3 + data transfer) vs Hetzner. When is each the right choice? Argue both ways.
4. Your ALB shows healthy but users in Europe report 3–4 s page loads; Dhaka users are fine. Server is in ap-southeast-1. Diagnose and propose fixes in order of impact (hint: physics + CDN).
5. From memory: draw the full AWS architecture for the final project (Route 53 → CloudFront → ALB → EC2 → RDS/Redis) and mark which subnet each component lives in.